

Classroom Attendance Prediction Using Academic Schedule and Historical Attendance Data

Submitted by: Aditya Suresh Chavhan

Roll No.: 2501032

1. What This Project Is About

This project is a **Classroom Attendance Prediction System** that uses historical attendance data and academic schedule information to understand attendance patterns and predict attendance for upcoming lectures.

The project considers different factors that can influence attendance, such as:

- Lecture date and day of the week
- Lecture number and timing
- Subject and subject code
- Faculty ID and faculty experience
- Semester, branch, and section
- Classroom
- Total enrolled students
- Previous lecture attendance
- Attendance history/rolling average
- Gap since the previous lecture
- Practical or theory lecture
- Internal test week

- Assignment due
- Holiday before/after
- Weather
- Special events

The system has **two machine learning objectives**:

1. **Regression** — predict the expected **Attendance Percentage**.
2. **Classification** — predict the **Attendance Category**.

The complete workflow is:

Dataset → Data Cleaning → Feature Engineering → Exploratory Data Analysis → Data Preprocessing → Model Training → Model Evaluation → Best Model Selection → Model Saving → Streamlit Deployment

What the Data Looks Like

The dataset (attendense.csv, cleaned into cleaned_attendance.csv) has 2,406 lecture records, spread across 273 different lecture-days, from 08 September 2025 to 13 August 2026. It covers the MCA course, Sections A and B, Semesters 1 to 3, 25 subjects, 22 teachers, and 2 classrooms (403 and 401/402).

What Each Record Contains

Each row has: Date, Day of Week, Lecture Number, Start/End Time, Subject, Subject Code, Faculty ID, Semester, Branch, Section, Classroom, Total Students, Students Present, Attendance Percentage, Previous Lecture Attendance, Gap Since Previous Lecture, Practical/Theory, Internal Test Week, Assignment Due, Holiday Before/After, Weather, Special Event, and Faculty Experience. That is 27 columns to start, and 39 columns after we added new features (see Section 3).

Privacy

No student names or roll numbers are in this dataset. Teacher IDs, subject codes, and classrooms are already coded, not written in full.

3. Cleaning the Data and Studying It

Cleaning Steps

- Missing values: Only the Faculty Experience column had missing values (167 out of 2,406 rows). We filled these gaps with the middle (median) value.
- Duplicate rows: We found 0 fully duplicate rows. We also checked more strictly (same Date, Lecture Number, Semester, Branch, Section) and still found 0 duplicates — every row is one real, separate lecture.
- Formula check: We checked that $\text{Attendance Percentage} = (\text{Students Present} / \text{Total Students}) \times 100$ for every row. All 2,406 rows matched — 0 errors.
- Date and time: Dates were read in DD-MM-YYYY format. Start and End times were used to work out how long each lecture was.

New Features We Added

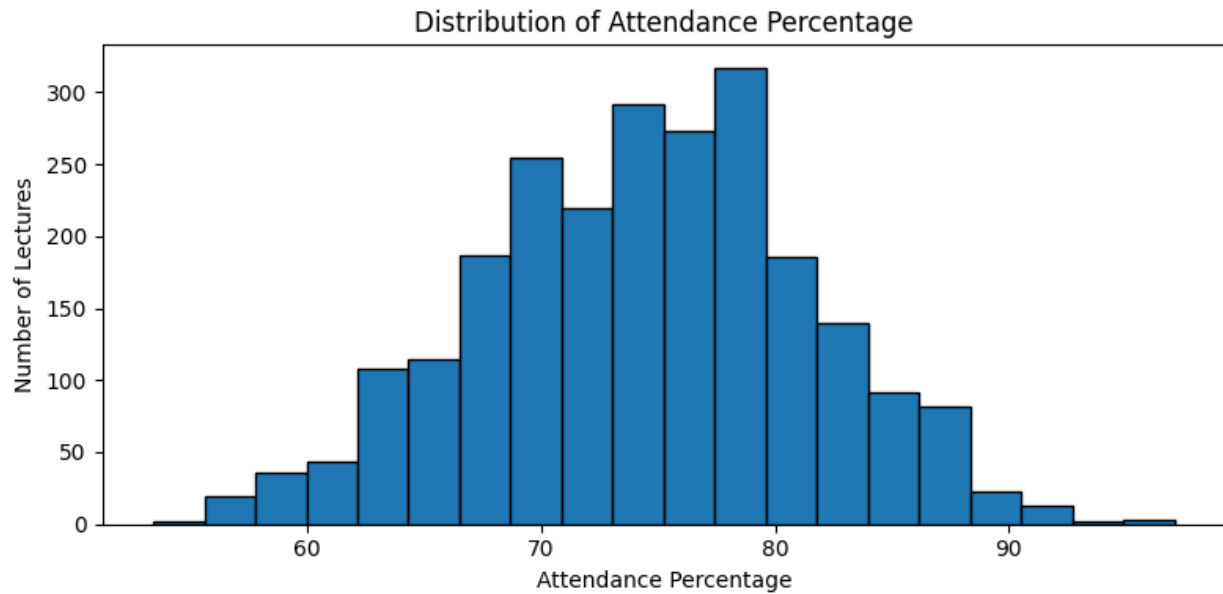
We added these extra columns to help the model make better predictions:

Feature	What It Means
Day of Semester	How many days since that semester's classes started
Week Number / Month	The week and month the lecture falls in
Previous Attendance Rolling Average	Average attendance of the last 3 lectures, for that Section
Consecutive Lecture Count	How many lectures in a row that day for that group
Days Since Last Holiday	How many days since the last holiday
Time Category	Morning or Afternoon, based on start time
Exam / Assignment / Holiday / Event Flags	Simple Yes/No columns turned into 1/0
Attendance Category	The label we predict: Low (below 50%), Medium (50–75%), High (above 75%)

Charts From the Real Data

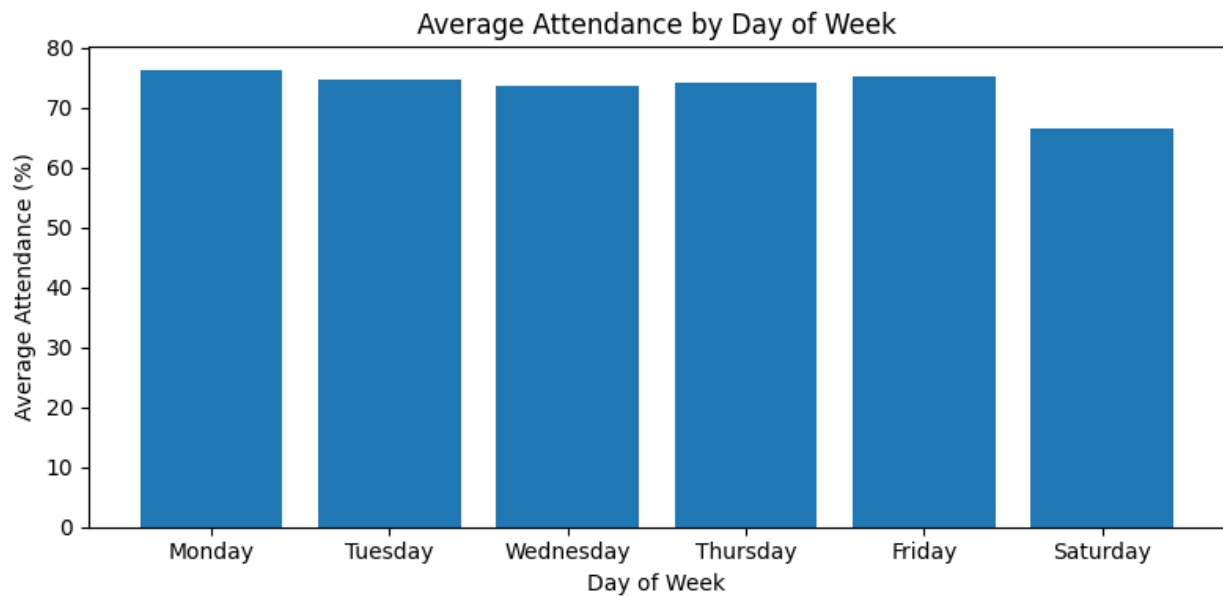
Every chart below comes straight from the real dataset (cleaned_attendance.csv). These are the actual results from running the project notebook — not example pictures.

How Attendance Is Spread Out



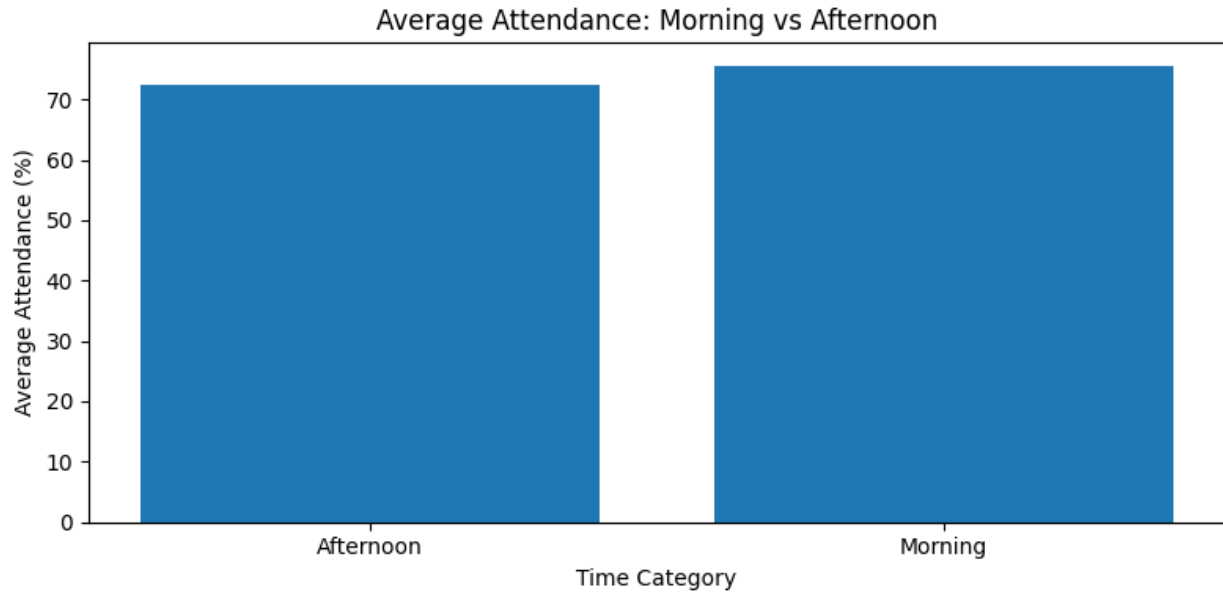
Attendance ranges from 53.4% to 97.1%. The average is 74.48%, across all 2,406 lectures.

3.3.2 Attendance by Day of the Week



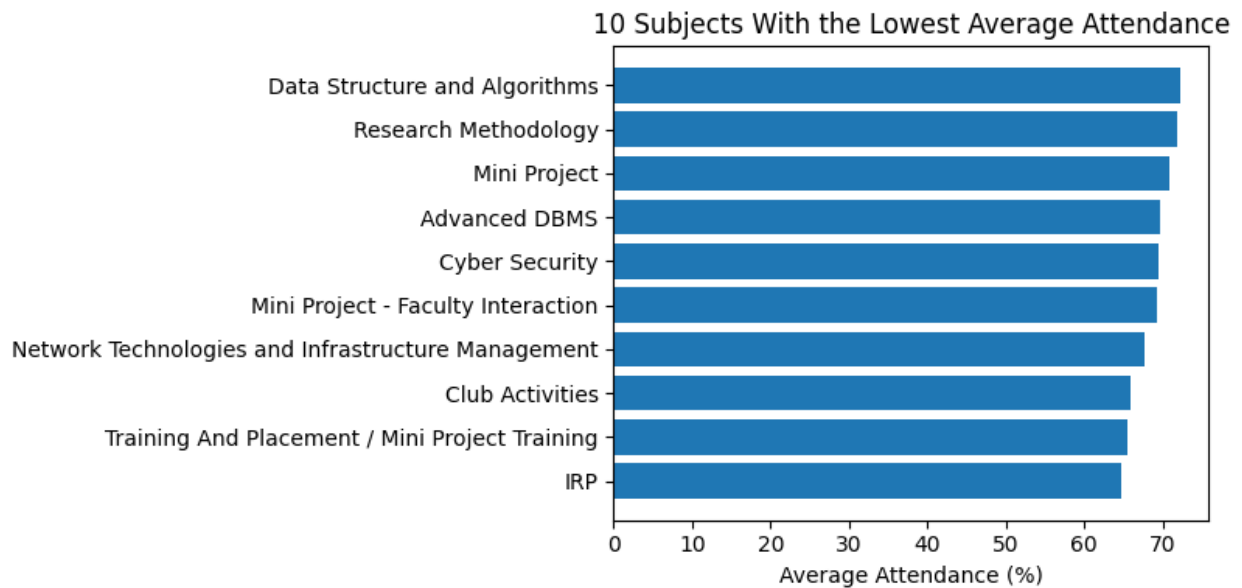
Saturday has the lowest attendance (66.4%). Other days are between 73.5% and 76.3%.

3.3.3 Attendance by Time of Day



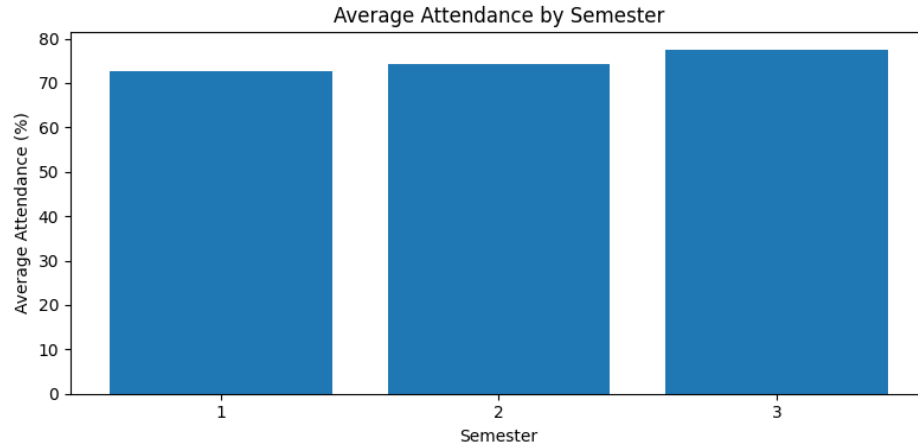
Comparing Morning lectures with Afternoon lectures, using the full dataset.

3.3.4 Ten Subjects With the Lowest Attendance

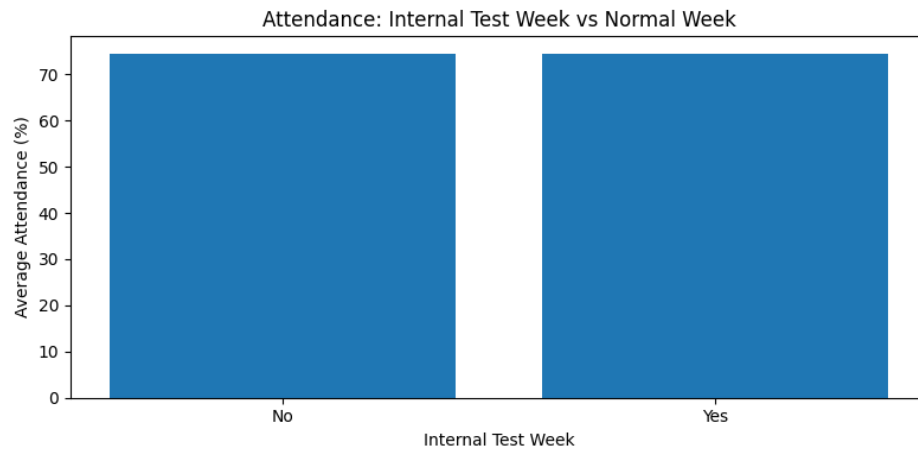


*IRP, Training & Placement / Mini Project Training, and Club Activities have the lowest attendance.
Hands-on practical classes usually have better attendance than these.*

3.3.5 Attendance by Semester

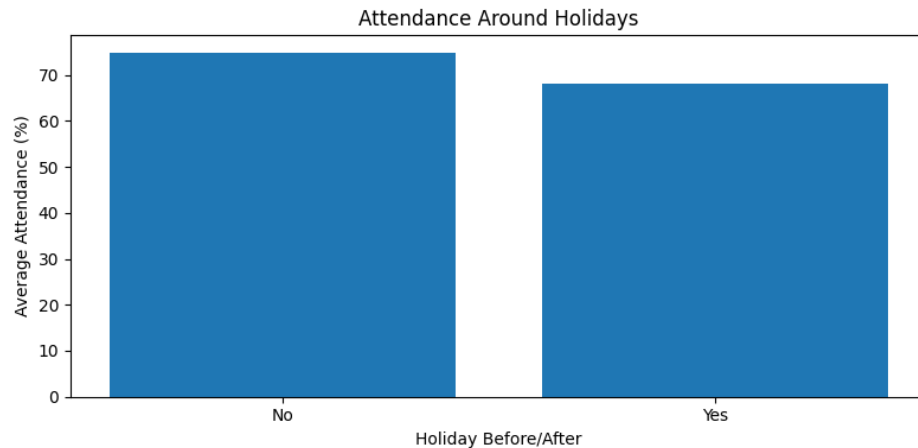


3.3.6 Attendance During Test Weeks



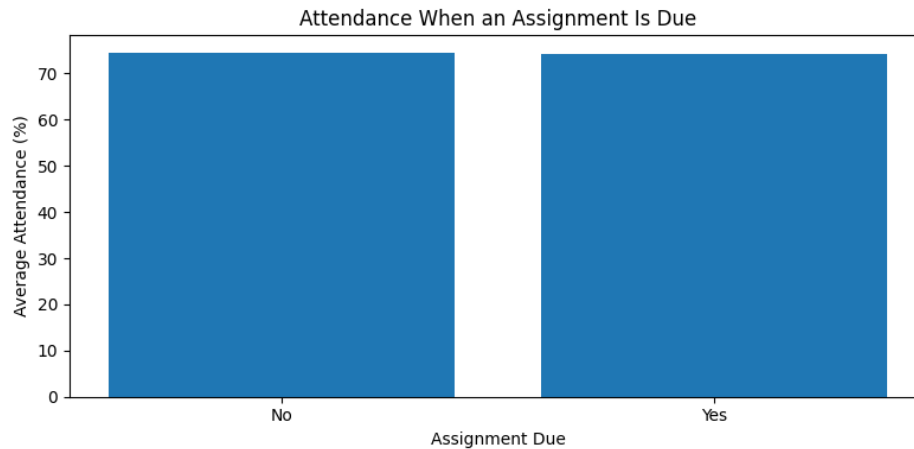
Almost no difference: 74.35% during test weeks vs. 74.49% at other times.

3.3.7 Attendance Around Holidays



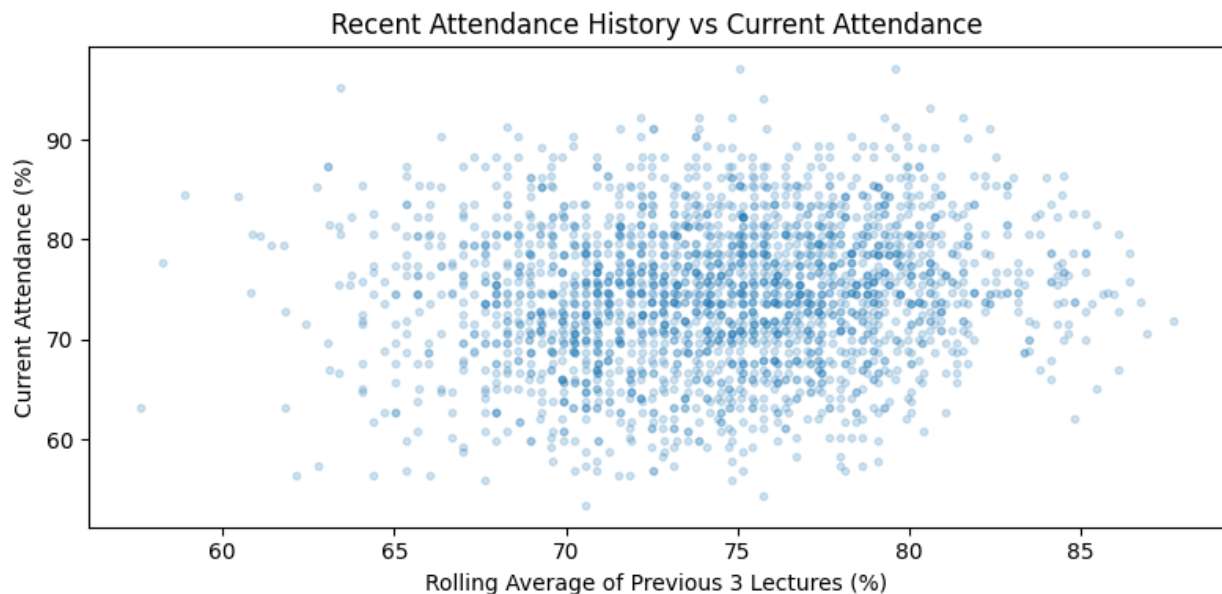
The biggest drop we found: 68.06% near holidays, compared to 74.86% normally.

3.3.8 Attendance When an Assignment Is Due



Almost no difference: 74.18% when an assignment is due vs. 74.51% otherwise.

3.3.9 Does Last Lecture's Attendance Predict This One?



A weak but real link (correlation = 0.19). This is a fair signal to use, because it only looks at what happened before the lecture, not after.

4. Building the Prediction Models

4.1 What We Are Predicting

- Number prediction (Regression): the exact Attendance Percentage.

- Category prediction (Classification): Low, Medium, or High. (No lecture in this data ever had attendance below 50%, so only Medium and High actually appear in the results.)

4.2 Splitting the Data for Testing

We split the data by time, not randomly — the model was trained on the earliest 80% of lectures (by date and time) and tested on the newest 20%. This way, the model is only ever tested on lectures that happen after the ones it learned from, which is closer to how it would really be used.

4.3 Preparing the Data for the Models

Number columns were filled in (if missing) and scaled. Text/category columns (like Subject or Day of Week) were filled in and converted into a number-friendly format (one-hot encoding). This exact same preparation step is built into every saved model, so nothing needs to be redone later.

4.4 Models We Trained

For the number prediction (Regression): Linear Regression, Decision Tree, Random Forest, Gradient Boosting, SVM, XGBoost.

For the category prediction (Classification): Logistic Regression, Decision Tree, Random Forest, SVM, k-Nearest Neighbors, Naïve Bayes, XGBoost.

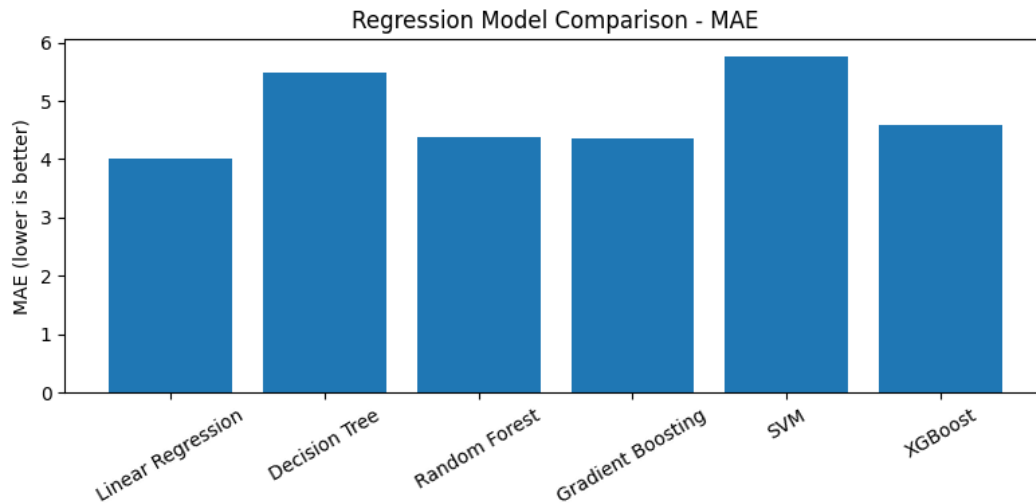
This matches every model type asked for in the project guidelines. (CatBoost was not used, since the guidelines list it as optional.)

5. Results — How Well the Models Worked

5.1 Number Prediction Results (tested on the newest 20% of lectures)

Model	MAE	RMSE	MAPE (%)	R ²
Linear Regression ★	4.0030	5.0717	5.2268	0.3065
Gradient Boosting	4.3456	5.4434	5.5609	0.2011
Random Forest	4.3705	5.4730	5.6034	0.1924
XGBoost	4.5857	5.7451	5.9991	0.1101
Decision Tree	5.4826	6.8747	7.1012	-0.2743
SVM	5.7635	7.0509	7.2328	-0.3404

(MAE, RMSE, and MAPE are error scores — lower is better. R² shows how well the model fits the data — higher is better, and 1.0 would be a perfect fit.)

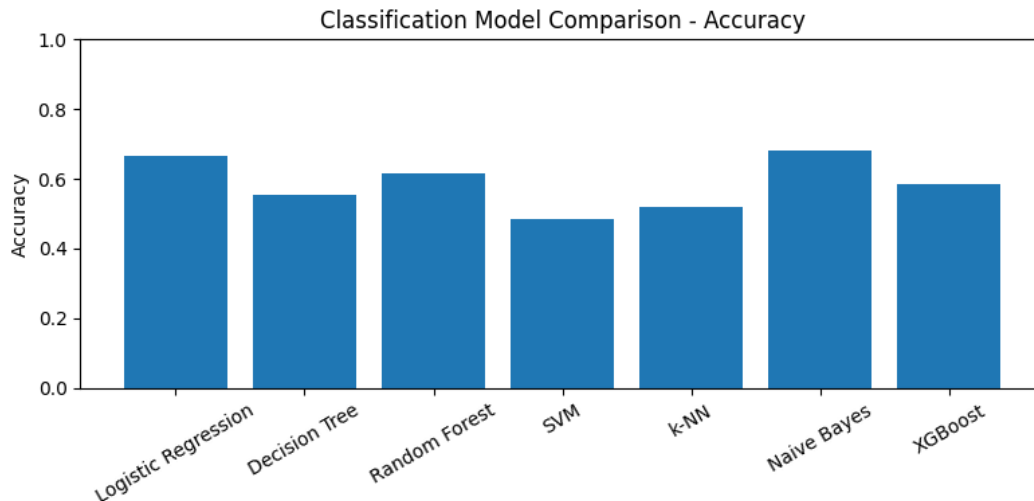


Error (MAE) for all six number-prediction models. Lower bars are better.

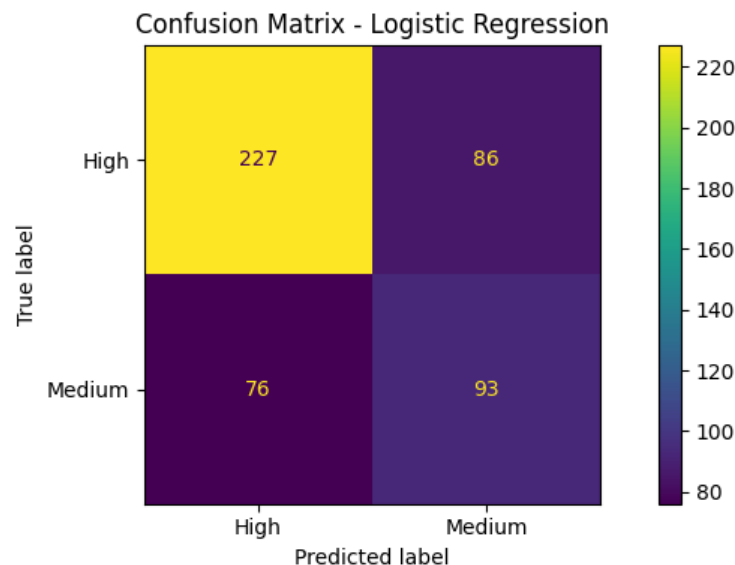
Linear Regression gave the best result (lowest error — MAE 4.00, meaning it was off by about 4 percentage points on average, or 5.2% in relative terms). This means the connection between our features and attendance is mostly simple and straight-line — the more complex models (Random Forest, XGBoost) did not do better, and even did slightly worse, on this test data.

5.2 Category Prediction Results (Medium vs. High)

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression ★	0.6639	0.6687	0.6639	0.6660	0.7045
Naïve Bayes	0.6826	0.6643	0.6826	0.6456	0.6802
Random Forest	0.6162	0.6600	0.6162	0.6249	0.6938
XGBoost	0.5851	0.6778	0.5851	0.5891	0.6514
Decision Tree	0.5539	0.5368	0.5539	0.5438	0.4919
k-NN	0.5187	0.5931	0.5187	0.5258	0.5523
SVM	0.4855	0.6001	0.4855	0.4782	0.6304



5.2.1 Confusion Matrix — Best Model (Logistic Regression)



6. The Dashboard We Built (app.py)

We built a Streamlit web app so anyone can use these models easily, without writing code. It can:

- Load the cleaned data and all 13 trained models (6 number-predictors + 7 category-predictors) at once, so no single model is forced as "the only one."
- Show a quick summary of the dataset — total records, columns, average attendance, number of subjects — and a sample of the actual data, with a clear note that the data is synthetic.
- Show charts: attendance spread, attendance by day of week, lowest-attendance subjects, and lecture slots with low attendance.

- Show how test weeks, assignments, holidays, and events affect attendance.
- Show live, data-based key findings (not hard-coded text) — like the correlation number and the share of low-attendance lectures.
- Compare all models side by side, with the best one for each task highlighted.
- Let the user fill in details for an upcoming lecture (date, time, subject, teacher, semester, section, room, expected attendance history, and conditions like test week or holiday) and instantly see a predicted attendance percentage and category from every model.

This covers what the project brief asked for in the final deployment stage — predicting future lectures, spotting weak time slots and subjects, and showing the effect of tests, holidays, and other conditions.

7. Main Findings, in Short

- A lecture's own attendance is weakly linked to the attendance of the lecture right before it (correlation 0.19) — a small but real, fair signal.
- No lecture in this dataset ever had attendance below 50%, so the "Low" category never shows up in the results.
- Holidays cause the biggest drop in attendance we found (68.1% vs. 74.9% normally), followed by special events (70.4% vs. 74.6%).
- Test weeks and assignment deadlines made almost no difference in this dataset.
- Saturday classes and non-teaching activities (IRP, placement training, club activities) have the lowest attendance.
- Best number-predictor: Linear Regression (average error about 4 points, R^2 0.31). Best category-predictor: Logistic Regression (F1 score 0.666, ROC-AUC 0.70).